

AI 时代的编程指南

看懂地图



Charles Tao 著，与 Claude 协作完成

books.stickmancharles.com · v1.2

正文 CC BY-NC-ND 4.0 · 代码 MIT

署名: Charles Tao, 《AI 时代的编程指南》, books.stickmancharles.com

AI 时代的编程指南 · 第一册《看懂地图》

你不需要成为程序员，但你需要看懂程序员的世界。

Charles Tao 著（与 Claude 协作完成，这本身就是本书讲的工作方式） v1.2 · 2026 年 8 月 · 系列共五册 · 本册难度 ★

献词

谨以此书献给我的父亲，Roy。

感谢他一直以来对我做项目的支持与肯定。也是他用实际行动告诉了我，只要想学，最好的时间就是现在。

目录

序 · 上限写在地图的边缘

第一部分 · 地图

1. 当你按下回车 · 一个网页的诞生
2. 前端与后端 · 舞台与后厨
3. 数据库 · 软件的记忆
4. API · 软件之间的握手
5. 代码住在哪里 · 服务器、云与 Serverless
6. 域名与 DNS · 互联网的门牌与电话簿

第二部分 · 工具 7. 终端 · 和电脑说话的第二种方式 8. Git 与 GitHub · 时间机器与云存档 9. 秘密与配置 · 环境变量、API Key 与 .env 10. 依赖 · 站在一亿个陌生人的肩膀上 11. 部署 · 从"我电脑上能跑"到"全世界都能用"

第三部分 · 心法 12. AI 编程的真相 · 你来当导演 13. 怎么提需求 · 从模糊的想法到 AI 一次做对 14. 验证与调试 · 不会检查，就不算做完 15. 安全 · 不懂的人不知道自己在裸奔 16. 持续变强 · 遇到不懂的词，你已经知道该怎么办

写在最后 · 给爸爸

附录 A · 术语表（中英对照） 附录 B · 解剖我们自己的项目 附录 C · 红线卡（危险动作清单）

序 · 上限写在地图的边缘

我们用 AI 编程，已经做出过真东西。跑在区块链上的合约，插在树莓派上的读卡器，被真实用户打开过的网站。按这份成绩单，我们应该越来越自信才对。

实际的感觉正好相反，天花板好像比别人低。AI 明明什么都会，我们却常常卡住。不敢动某个文件，看不懂它在终端里敲的命令，听到"要不要加个 serverless function"时只能说"嗯，你觉得好就行"。这种感觉，爸爸比我更强烈。

卡在哪里，我想了很久。智力够用，工具也都在手边，我们缺的是一张地图。

AI 是历史上最快的车，路牌上写的却是 DNS、数据库、环境变量、部署。看不懂路牌的人，只敢在家附近打转。上限被压低的感觉就是这么来的。车没有问题，我们还没学会看路。

所以这本书不教你写代码，那件事 AI 已经做得比绝大多数人好了。这本书教的是看懂。软件世界是怎么搭起来的，AI 每一步在做什么，那些反复出现的词到底指什么。看懂了，你就敢往更远的地方开。

全书分三部分。第一部分是地图，网站、数据库、服务器、DNS，这些东西各自是什么，怎么连在一起。第二部分是工具，终端、Git、环境变量、部署，你每天都在用，却可能从没被人正经介绍过。第三部分是心法，怎么给 AI 提需求，怎么验证它的工作，安全上又该守住哪几条线，这部分有我们自己付过学费的教训。

建议的读法是每晚一章，一章十分钟以内。读到不懂的词，翻附录 A。读完一章，花五分钟在我们自己的项目里把它找出来。概念只有在你自己的东西里出现过，才真正属于你。

爸爸 47 岁开始学这些，我 14 岁开始学这些，我们坐在同一张桌子前。上限这个东西，从来不写在年龄里，只写在地图的边缘。

而地图，是可以画大的。

第一部分 · 地图

第 1 章 当你按下回车 · 一个网页的诞生

打开浏览器，输入一个网址，按下回车。接下来的半秒钟里，发生了一连串你从未见过的事。

浏览器先去问一个叫 DNS 的系统："这个名字对应的是哪台电脑？"DNS 是什么，第 6 章细讲，先记住它是个翻译名字的电话簿。拿到答案，浏览器找到了地球某处一台永远开着机的电脑，服务器（Server）。你的浏览器在这场对话里叫客户端（Client），它向服务器发出一个请求（Request）："把这个页面给我。"服务器回一个响应（Response），里面装着几种文件。HTML 是页面的骨架，CSS 是皮肤和化妆，JavaScript 是动作和交互。浏览器把它们画出来，就是你看到的网页。

拿点外卖来想这件事。你是客户端，餐厅是服务器。下单是请求，出餐是响应。双方约好的沟通格式，单子怎么填、餐怎么送，就是 HTTP。你常看到的 HTTPS 多出来一个 S，是加密（Secure）的意思，相当于把信装进只有你们俩能打开的信封，中途谁也偷看不了、篡改不了。现在正经网站都必须是 HTTPS。

这一章只想让你记住一件事，整个互联网就是无数次一问一答。你以后做的每一个功能、调的每一个接口，都是在扮演提问的一方，或者回答的一方。想清楚谁在问、谁在答、问的是什么，很多看起来复杂的东西会突然变简单。

试一试

随便打开一个网页，按 F12 打开开发者工具，切到 Network（网络）面板，刷新页面，你会亲眼看到那半秒钟里的每一次请求和响应。

第 2 章 前端与后端 · 舞台与后厨

一个软件通常分成两半，跑在两个完全不同的地方。

前端 (Frontend) 是跑在用户设备上的那一半，浏览器里的页面，手机上的界面。它由上一章说的 HTML、CSS、JavaScript 组成。你听过的 React，是一种把界面做成积木来拼的流行框架，我们自己的网站就是用它搭的。

后端 (Backend) 是跑在你的服务器上的那一半，处理登录，读写数据库，验证权限，调用区块链。用户永远看不见它，但它掌管一切规则。

为什么要分成两半？用餐厅来想。前厅的菜单、桌椅、装修是前端，顾客看得见摸得着，甚至可以拿笔在菜单上乱涂。后厨和收银系统是后端，菜怎么做，账怎么算，轮不到顾客插手。关键在这里，前端的一切都在用户手里，用户可以看到它的全部代码，也可以随意篡改它。所以任何真正重要的东西，密码验证、金额计算、权限判断、数据库钥匙，都绝对不能放在前端。

这就引出了本书最值钱的一句话，永远不要相信前端。顾客说"我刚才已经付过钱了"，你也得去收银系统里查记录，光听他说不行。无数真实事故，都源于有人把秘密或规则放在了用户碰得到的地方。

放到我们自己的项目里看，用户看到的上传页面是前端。真正验证文件、决定要不要在链上铸造凭证的逻辑，是后端。界面可以随便改，规则必须锁在后厨。

试一试

在任意网页上右键选"查看网页源代码"，你看到的就是前端的全部。体会一下"用户能看到前端的一切"是什么意思。

第 3 章 数据库 · 软件的记忆

程序有个天生的毛病，一关掉，脑子里的东西就全没了。可用户第二天回来，凭什么你还记得他是谁、他传过什么文件、他有哪些凭证？靠的是数据库（Database），一个专门负责记住的软件。这些东西从来不存在程序里，程序只在需要时去数据库里取。

最快的理解方式是把数据库想成一个极其严谨的 Excel。一张表（Table）对应 Excel 里的一个工作表。一行（Row）是一条记录，比如一个用户。一列（Column）是一个属性，比如邮箱、注册时间。我们用的 Supabase，底下就是世界上最受欢迎的开源数据库 Postgres。

和数据库说话有专门的语言，叫 SQL。别被吓到，它的核心只有四个动词，查（SELECT）、增（INSERT）、改（UPDATE）、删（DELETE）。比如这一句。

📄sql

```
SELECT * FROM users WHERE email = 'dad@example.com';
```

意思是从 users 表里，把邮箱等于这个的行都找出来。你看 Claude Code 写的 SQL，八成都是这四个动词的变体。

那为什么不直接用 Excel 或者一堆文件来存？数据库解决了四个 Excel 搞不定的问题。一千个人同时读写不打架。在百万行里瞬间查到你要的那一条。精确控制谁能看哪些行、谁能改哪些列。断电、崩溃也不丢数据。至于 Supabase 是什么，它帮你把 Postgres 托管好，再送你登录系统（Auth）、文件存储和现成的 API，所以 Claude Code 三句话不离它。

顺带认识一个你常会见到的词，迁移（Migration）。每次修改表结构，加一列，建一张表，都写成一个脚本保存下来，让数据库的每一步变化都有据可查、可以重放，相当于给数据库结构做版本记录。

关键领悟

你产品里一切属于用户的東西，几乎都住在数据库里。设计表结构的过程，就是想清楚我的产品到底要记住什么。

试一试

打开 Supabase 后台的 Table Editor，随便点开一张表，亲手把表、行、列各自指认一遍。

第 4 章 API · 软件之间的握手

软件不光和人打交道，更多时候是软件和软件打交道。它们之间的服务窗口，叫 API（Application Programming Interface，应用程序接口）。

用银行来想。金库（数据库）你不能进，你只能到柜台窗口（API），按规定格式填单子，办理规定范围内的业务。天气 App 显示的温度，是它去调了气象服务的 API。我们的网站给用户发邮件，是去调了 Resend 的 API。查一笔链上交易，是去调了区块链节点的 API。现代软件的真相，就是把十几个别人的 API 粘在一起，再加上你自己独有的那一点逻辑。

窗口的地址和规矩，有一套最流行的风格叫 REST。URL 是名词，表示哪种东西，比如 `/certificates/123` 指编号 123 的那张凭证。HTTP 方法是动词，表示想干什么，GET 是读，POST 是新建，PATCH 是修改，DELETE 是删除。一个具体的地址，比如 `GET /certificates/123`，叫一个端点（Endpoint）。

软件之间传的单子，用一种通用格式写，叫 JSON。长这样。

📄 json

```
{ "name": "Charles", "age": 14, "projects": ["EchoForge", "ForgeCard"] }
```

就是 **名字：值** 的组合，人能看懂，机器也能看懂。你在日志里、报错里、AI 的输出里会天天见到它。

最后认识 API Key，柜台发给你的通行证。它证明是我在办业务，服务商也靠它记账、限流。所以它是秘密，拿到你 key 的人，就能冒充你、花你的钱。怎么保管它，第 9 章专门讲。

关键领悟

会读 API 文档（或者让 AI 读给你听），等于拥有了全世界的乐高积木。

试一试

让 AI 列出我们某个项目实际调用了哪些外部 API，各自负责什么，你会发现"粘乐高"这个说法一点不夸张。

第 5 章 代码住在哪里 · 服务器、云与 Serverless

代码写好了，总得在某台电脑上跑起来。跑在哪，是个层层递进的故事。

第一层，你自己的电脑。开发时代码就跑在本机，浏览器里那个 `localhost` 就是我自己这台机器的意思。它有两个致命问题，你一合上电脑，网站就死了，而且别人根本访问不到你。

第二层，租一台云上的电脑。所谓云（Cloud），就是别人机房里成千上万台永远开机的电脑，租给你用。你可以租一整台（常听到的词是服务器或 VPS），24 小时开机，自己装环境，自己维护。像租了间毛坯房，自由，但水电煤气全得自己操心，而且不管有没有客人，房租照付。我们在 DigitalOcean（常简称 DO）上就租着这样一台，第四册会带你亲自登上去看看。

第三层，Serverless（无服务器）。这个名字起得很坏，容易让人以为真的没有服务器。服务器还在，只是全归平台管，你不用碰。你只把一段函数交给平台，有人访问时，平台临时把它唤醒，跑完，再睡下，按运行次数计费。像共享厨房，有单才开火，没单不付房租，厨房卫生归平台管。你听到的 Function / Edge Function，就是这种被叫醒才干活的小段代码。

第四层，静态托管。你要放的只是前端文件（HTML/CSS/JS）的话，连函数都不用，平台直接把文件复制到全球各地的节点（这套分发网络叫 CDN），谁访问就近取货，像自动售货机。

现在可以一句话说清 Vercel 是什么了。它把你的前端做成全球售货机，把你的后端函数做成共享厨房，还连着 GitHub 自动上新（第 11 章讲）。我们的网站部署在它上面，原因就是那个。

那什么时候还需要一台永远醒着的真机器？当你的程序必须一直跑着的时候，比如持续监听区块链的程序。顺便说个好玩的，ForgeCard 那台插着读卡器的树莓派，就是我们自己客厅里的一台小服务器，永远醒着，等卡片靠近。

关键领悟

代码住哪，取决于它多久醒一次。偶尔醒，Serverless。永远醒，服务器。只是被看，静态托管。

第 6 章 域名与 DNS · 互联网的门牌与电话簿

每台联网的电脑都有个真实门牌号，叫 IP 地址，一串类似 `76.76.21.21` 的数字。机器认它，人记不住它。于是人类发明了域名（Domain），给门牌号起个名字。

把名字翻译回号码的系统，就是 DNS（Domain Name System）。它像全世界共用的一本联系人电话簿，你存的是"家"，打电话时手机自动翻译成号码。第 1 章那半秒钟里的第一步，就是查这本电话簿。

买域名，买的其实是每年付费租用这个名字的权利，卖家叫注册商。买完之后，你要往电话簿里写几条 DNS 记录，最常见四种。A 记录把名字指向一个 IP，等于说这个名字住在这台机器。CNAME 把名字指向另一个名字，找它就等于找它，把域名指到 Vercel 用的就是这种。TXT 像在名片上贴张便签，最常见的用途是证明这个域名归我，所以 Resend、Google 这类服务让你验证域名时，都是让你加 TXT 记录。MX 负责说清发给这个域名的邮件该往哪送。

顺带认识一下我们家的电话簿管理员，Cloudflare。我们的域名记录就写在它那里。除了当电话簿，它还兼任门口的调度员和保安（CDN 分发与安全防护），它还能干的那些事，第四册专门讲。

两个每个人都会撞上的现象。其一，改了 DNS 为什么不马上生效？全世界有无数份电话簿的缓存副本，要按各自的有效期（TTL）陆续更新，这个过程叫传播，从几分钟到一两天不等。其二，HTTPS 的锁是哪来的？网站需要一张证书来证明身份并加密通信。好消息是 Vercel 这类平台现在都自动帮你申请续期，你只要知道那把锁的含义就行。

关键领悟

DNS 出问题的症状千奇百怪，一部分人打得开一部分人打不开，刚改完不生效，邮件收不到。以后遇到这类玄学，第一反应应该是电话簿还没更新完，或者哪条记录写错了。

试一试

搜一个"DNS lookup"在线工具，输入我们自己的域名，看看它背后到底写了哪几条记录，一条条认出来。

第二部分 · 工具

第 7 章 终端 · 和电脑说话的第二种方式

图形界面是指指点点，终端（Terminal）是直接下命令。同一台电脑，两种说话方式。程序员偏爱终端，图的是精确、可重复、可自动化，一条命令可以原样再跑一遍，一百条命令可以写成脚本，酷只是副产品。也正因为如此，AI 也爱它。不管你是在 VS Code 的 Claude 插件里，还是在 Claude Code 网页版里派活，AI 真正动手时，走的都是终端命令，而且每一条都当着你的面敲。我们自己确实很少亲自开终端，但这恰恰是要读这一章的原因。你可以不常说这门语言，但你必须听得懂 AI 在用它说什么。

看得见，就应该看得懂。高频命令其实只有一小把。

命令	意思	记法
<code>pwd</code>	我现在在哪个文件夹	print working directory
<code>ls</code>	这里有什么	list
<code>cd 路径</code>	进入某个文件夹	change directory
<code>mkdir 名字</code>	新建文件夹	make directory
<code>cp 甲 乙</code>	复制	copy
<code>mv 甲 乙</code>	移动/改名	move
<code>cat 文件</code>	把文件内容打出来	concatenate 的缩写
<code>grep 词 文件</code>	在文件里搜这个词	老工具的名字，记"搜"就行
<code>rm 文件</code>	删除 ⚠️ 没有回收站	remove

再补两个概念。路径分绝对的（从根写起，`/Users/charles/...`）和相对的（从当前位置写起，`./src/...`），`~` 代表你的家目录。竖线 `|` 叫管道，把前一条命令的输出喂给后一条，像流水线接力。

终端里真正危险的东西极少，基本都是删除类。看到 `rm -rf`（递归强制删除）出现在 AI 要执行的命令里，停下来多看一眼再放行。除此之外，终端不可怕，它只是把菜单背下来的点菜方式。

关键领悟

读完这本书，你应该能看懂 Claude Code 终端输出的百分之八十。剩下的百分之二十，问它就好。

试一试

打开终端，用 `pwd`、`ls`、`cd` 逛一逛自己的电脑，最后 `cd` 进我们某个项目的文件夹，`ls` 看看里面的结构。

第 8 章 Git 与 GitHub · 时间机器与云存档

两个古老的噩梦，改崩了回不去，和两个人同时改互相覆盖。Git 就是为此发明的，一台给代码用的时间机器。

打游戏你就懂了。`commit`（提交）是存档点，每个存档附一句话，说明这次改了什么。`branch`（分支）是开一个平行存档去试危险打法，主线不受影响。试成了，`merge`（合并）回主线。试崩了，整条分支扔掉，无事发生。想回到任何一个存档点，随时可以。

GitHub 则是云存档加联机大厅，把你的 Git 仓库（repository，简称 repo）放到云上。Git 是工具，GitHub 是网站，关系就像邮件和 Gmail。`push` 是把本地存档传上云，`pull` 是把云上的新存档拉下来，`clone` 是把整个仓库原样抄一份到新电脑。多人协作时你还会见到 PR（Pull Request），意思是“我改好了一版，请你看过再合并”，AI 帮你改代码时走的也常是这个流程。

现在你明白 Claude Code 为什么老在 `commit` 了。AI 也需要存档点。它每完成一小步就存一次档，一旦搞砸，回滚就行。经历过改坏了却回不去的人（比如我们），会真心爱上这个设计。

平时你只需要三招傍身。`git status`，我现在改了哪些还没存档。`git log --oneline`，存档历史一览。`git diff`，这次到底改了什么。让 AI 帮你回滚时，你能看懂它在做什么，用不着闭着眼睛祈祷。

关键领悟

`commit` 越勤，胆子可以越大。没有版本控制的实验是赌博，有版本控制的实验才是科学。

试一试

在我们任意一个项目里跑 `git log --oneline`，读读最近二十条存档说明，那就是这个项目的编年史。

第 9 章 秘密与配置 · 环境变量、API Key 与 .env

有些东西永远不能写进代码。数据库密码，API Key，钱包助记词。代码会进 Git，会上 GitHub，会被 AI 反复读来读去。代码天生是要流传的，秘密天生是不能流传的。

解决办法是把秘密放在代码外面的口袋里，代码运行时伸手去摸。这个口袋就是环境变量（Environment Variable）。本地开发时，习惯把它们集中写在一个叫 `.env` 的文件里，再在 `.gitignore`（一份“Git 请忽略这些文件”的清单）里写上 `.env`，确保这个口袋永远不会被拍进存档。线上呢？部署平台（Vercel、Supabase）都有专门的设置页面让你填环境变量。同一份代码，本地穿一件马甲，线上穿另一件，口袋里装着不同的钥匙。这也是开发环境和生产环境这两个词的一半含义，同样的代码，不同的口袋。

判断一个东西算不算秘密，就问一句，陌生人拿到它会怎样？能花你的钱、动你的数据、冒充你的身份，那就是秘密，进环境变量。只是个公开的项目地址？那不算。

万一泄露了怎么办？记住一条反直觉的铁律，去服务商后台立刻作废（revoke）并换新（rotate）那把钥匙，光把那行代码删掉没有任何用。Git 是时间机器，历史存档里它还在。凡是曾经出现在 GitHub 上的秘密，一律当作已经泄露处理。

关键领悟

秘密管理是一种习惯。养成了，几乎零成本。没养成，一次事故就够你疼一年。

试一试

打开我们项目里的 `.env` 文件（还有 `.gitignore`），认一认里面每一把钥匙分别开哪扇门。不确定的，问 AI。

第 10 章 依赖 · 站在一亿个陌生人的肩膀上

没有人从零写一切。处理日期，发请求，画图表，这些别人早就写好、打磨过千百遍的轮子，叫库（Library）或包（Package）。把它们装进你项目的工具，叫包管理器，JavaScript 世界用 `npm`，Python 世界用 `pip`，像是代码的应用商店。你装进来的这些东西，统称你的依赖（Dependencies）。

打开我们任何一个前端项目，都能看到两样东西。一个是 `package.json`，项目的购物清单，写着依赖哪些包，各要什么版本。另一个是 `node_modules` 文件夹，按清单搬回家的货。它大得离谱是正常的，删了随时能凭清单重装（`npm install`），而且永远不需要手动去改它。

版本号 `1.2.3` 这三段数字有讲究，这套约定叫 `semver`。第一段变了，可能不兼容旧用法。后两段变，一般是加功能和修 bug。"在我电脑上能跑，在你电脑上崩了"的惨案，很大一部分来自两台电脑装的版本不一致。所以还会有一个 `lock` 文件（如 `package-lock.json`），把每个包的精确版本钉死，保证人人装到的完全一样。

依赖也有代价，知道两句话就够。包会过时腐烂，多年没人维护。包也可能被坏人下毒，这叫供应链攻击。所以当 AI 说"我装个包来解决这个问题"时，你现在完全知道它在干什么了，偶尔可以追问一句，这个包多少人在用？最近还在更新吗？

关键领悟

现代开发的默认姿势就是站在别人肩膀上。要紧的是知道自己站在谁的肩膀上，用不着拒绝依赖。

试一试

打开我们项目的 `package.json`，挑三个没见过的依赖，让 AI 一句话解释各自是干嘛的。

第 11 章 部署 · 从"我电脑上能跑"到"全世界都能用"

部署 (Deploy)，就是把代码放到全世界访问得到的地方，让它跑起来。听起来吓人，但我们其实每周都在做，只是它被自动化得太顺滑，顺滑到你没意识到自己在做一件从前很难的事。

一次典型的现代部署是这样走的。你（或 AI）`git push` 到 GitHub。Vercel 收到通知，拉取最新代码，执行构建 (Build)，把源代码翻译、打包、压缩成可以上线的成品。成品被分发到全球节点，上线完成。整条流水线全自动。"我 push 了一下，网站就更新了"，全部原理就是这个。

一个常见困惑，本地明明好好的，为什么一部署 build 就失败？几乎总是三个原因之一。线上缺环境变量，第 9 章的口袋没配。依赖版本不一致，第 10 章的清单没钉死。或者代码里有本地被宽容、线上被严查的错误。处理方式也固定，把构建日志丢给 AI 看，你看 AI 的结论对不对。

再认识一个宝藏概念，环境不止一个。`localhost` 是你的私人试验田。Preview (预览部署) 是 Vercel 给每一次改动单独生成的临时网址，正式上线前，先在预览网址上点一遍。Production (生产环境) 才是真实用户访问的那个。顺序永远是本地、预览、生产。万一生产出了问题，还有最后的安全网，回滚 (Rollback)，一键退回上一个正常版本。为什么能回滚？第 8 章说过，每一版都有存档。

关键领悟

部署是一条有存档、有预览、能回滚的流水线，用不着当成一次惊险的跳跃。理解了流水线，你就再也不怕上线两个字。

试一试

打开 Vercel 后台，找到我们项目的 Deployments 列表，每一行就是一次部署，点开看看它对应哪一个 commit。

第三部分 · 心法

第 12 章 AI 编程的真相 · 你来当导演

Claude Code 这类工具在做什么？拆开看是一个循环。读你的代码库，这些进入它视野的信息叫上下文（Context）。理解你的意图，定计划，执行命令、修改文件，观察结果，再来一轮。它的超能力是知识面广、不知疲倦、能同时读几十个文件。它的弱点也真实存在。会自信地犯错。上下文有限，对话太长会忘记前面的约定。倾向于顺着你说“你说得对”。还有最要命的一条，它对你独有的约束一无所知，除非你告诉它。

最后这条，我们付过真金白银的学费。有一次，一个看起来无害的重命名任务，AI 从头到尾执行得又快又漂亮。但它不知道我们有一份已经部署上链、部署后就无法修改的合约，必须和代码库保持一致。结果两边对不上，我们花了好几天做链上考古，才把现场拼回来。这件事没有让我们少用 AI。它教会我们的是，凡是已部署的、花了真钱的、不可逆的东西，红线必须由人来划，全世界只有你知道你的红线在哪里。后来我们把这些红线写成了给 AI 看的规矩，每次动那个项目前先让它读。这个做法，推荐给每一个用 AI 做真东西的人。

所以你的角色是导演。导演做三件事。给剧本，把需求讲清楚，下一章讲。划红线，什么绝对不能动。看回放，验证结果，第 14 章讲。至于打字，让 AI 去。

关键领悟

AI 把动手的成本降到了接近零，于是想清楚成了唯一值钱的事。

第 13 章 怎么提需求 · 从模糊的想法到 AI 一次做对

同一个 AI，有人指挥它像开挂，有人指挥它像抽奖。差别几乎全在需求怎么提。

先看对比。差的版本就一句，"给管理后台加个搜索"。好的版本是下面这样。

□text

在管理后台的凭证列表页顶部加一个搜索框：

- 按凭证编号精确匹配，按用户邮箱模糊匹配
- 输入时即时过滤，无结果时显示"未找到，请检查拼写"
- 样式跟现有筛选器保持一致
- 不要动后端接口

前者 AI 要靠猜，猜错一次你返工一次。后者它一次就能做对。

好需求有个万能骨架，四块。

- 背景，我在做什么，相关代码在哪。
- 目标，做成什么样，怎么算成功。
- 约束，不能动什么，必须用什么，红线在哪。
- 例子，给一组输入和期望输出，一个好例子顶三段描述。

不必每次都写全，卡壳时按这四块补齐，几乎总能解开。

再给两个立刻能用的招。第一招，大活拆小。别说"把整个网站重构了"，说"第一步，先把登录页拆出来，做完我看一眼再继续"。一次一个可验证的小步，AI 和人一样，步子太大容易扯着。第二招，先要计划，再要代码。跟它说"先告诉我你打算怎么做，列出要改哪些文件，先别动手"。看过计划再放行，能免掉一半返工，计划里就能看出它有没有误解你。

关键领悟

提需求的本质，是把你脑子里的画面传进 AI 脑子里。画面越清晰，成品越像。

试一试

把你最近一次让 AI 返工两次以上的需求找出来，用背景、目标、约束、例子四块重写一遍，对比一下差距。

第 14 章 验证与调试 · 不会检查，就不算做完

AI 说"完成了"，不等于完成了。它是真诚的，但真诚和正确是两回事。所以每次交付，按这个金字塔查一遍，从底往上。能跑吗，起码不报错。功能对吗，按需求逐条过。边界情况呢，空输入、连点两次、断网、超长文本。真实用户会遇到什么，换台设备、换个浏览器试试。大多数翻车翻在第三层，正常路径谁都会做对，边界才见功夫。

出错了先别慌，报错信息是礼物。一条报错通常包含三样，哪个文件第几行，什么类型的错，还有一句人话描述。你甚至不用全看懂，把报错原样完整地贴给 AI，比你转述"它好像坏了"有用十倍。日志 (Logs) 同理。本地看终端，线上看 Vercel 的 Functions 日志和 Supabase 的日志面板，线上出问题时，答案几乎都躺在日志里。程序员用了五十年的最土也最有效的探照灯，叫 `console.log` (Python 里是 `print`)，在可疑的地方打一盏灯，把变量的值打出来看，猜测立刻变成证据。

两个高级但救命的习惯。一是最小复现，想办法把 bug 关进最小的笼子，比如"只要输入是空的，点提交就崩"。定位到这一步，问题就解决了八成。二是换个会话重来。AI 在一条错误假设上狂奔太久时，别在原对话里纠缠，开个新会话，把问题重新描述一遍，常常瞬间解开。实在不行，睡一觉，这是正经的工程实践。

关键领悟

调试的本质是把猜测变成证据。一次只改一个变量，你就是科学家。

试一试

下次报错，先自己读一遍最后十行，猜一个原因，再贴给 AI，对答案。猜中率会一周比一周高。

第 15 章 安全 · 不懂的人不知道自己在裸奔

先说清楚为什么单开一章。"能跑"和"安全"看起来一模一样。不安全的网站照样能打开、能注册、能用，直到出事那天。AI 时代人人能做产品，最容易出的事故就在这里。好消息是，保命的底线只有五条，全是常识级别。

1. 秘密进环境变量，永不进代码（第 9 章）。泄露了就立刻作废换新，别幻想删掉那行代码有用。
2. 永远不要相信前端和用户输入（第 2 章）。一切验证、权限、金额计算，必须在后端再做一遍。
3. 数据库必须开行级权限。Supabase 里这叫 RLS（Row Level Security），给每张表立规矩，谁能看哪些行，谁能改哪些行。一张没开 RLS 的表，约等于向全网公开。让 AI 建表时，永远记得追问一句："RLS 配了吗？规则是什么？"
4. 永远不要自己发明登录系统。密码的存储、找回、防爆破，是无数人踩坑几十年的成果。用平台现成的 Auth（比如 Supabase Auth），别自己存密码，更绝对不能存明文。
5. 不可逆操作，上线前人肉确认。转账、铸币、群发邮件、删生产数据，这类动作在真实执行前，必须有一个人类看过并说"确认"。先发给自己，再发给用户。

每一条背后都有真实世界里成堆的事故。写死在代码里的 key，被扫库机器人几分钟内盗刷。只在前端校验价格，被人把 100 元改成 1 元下单。没开 RLS 的用户表，被整表拖走。这些都上过新闻，用不着我编。

关键领悟

安全是和出门锁门同级的生活习惯，五条底线全是常识。这一章值得每半年重读一遍。

试一试

现在就去 Supabase 后台，逐张表检查 RLS 是否开启。这可能是全书性价比最高的十分钟。

第 16 章 持续变强 · 遇到不懂的词，你已经知道该怎么办

这本书没法覆盖所有词，地图永远比领土小。但你现在拥有一台学习飞轮。造东西，撞见新词，花五分钟搞懂它，回去继续造。重点是那五分钟怎么花。三个问题，按顺序问 AI。

第一问："用 14 岁能懂的方式解释 X，再说清它解决了什么问题。"一个概念的灵魂在它解决的问题里，光背定义没有用。第二问："X 和 Y 有什么区别？什么时候用哪个？"新词几乎总是老词的邻居，放在一起才立体。第三问，也是最厉害的一问："我们的项目里有 X 吗？在哪？"这一问把抽象概念钉进你自己的代码库。在别人的例子里见过一百次，不如在自己的项目里指认一次。

然后，把这个词加进附录 A。那份术语表没有在我写完时定稿，它是活的，每征服一个新词就添一行。它长多大，你的地图就长多大。

最后想说清这本书的目标。目标里没有"成为程序员"这一项，那个职业本身正在被 AI 重新定义。这本书想把你带到的位置，是能和任何程序员、任何 AI 平等对话。你未必亲手砌墙，但你看得懂图纸，验得了质量。到那时，上限被压低的感觉会彻底消失。压低上限的从来只是几十个没对上号的名词，而名词，是全世界最容易学的东西。

关键领悟

不懂的词都能用三问打开。它解决什么问题？和邻居有什么区别？在我们的项目里的哪儿？

写在最后 · 给爸爸

爸爸：

感谢您一直以来对我的支持与鼓励。没有您，这本书、这个网站，甚至 Stickman Charles 都不会出现。

未来的变化会天翻地覆，但是我相信，只要不断学习，就已经在准备的路上。

—— Charles

附录 A · 术语表（中英对照）

这份表是活的，每征服一个新词，就自己加一行。

网络基础

- 客户端 (Client)：发起请求的一方，通常是用户的浏览器或 App。
- 服务器 (Server)：响应请求的一方，一台永远开机、等人来问的电脑。
- 请求 / 响应 (Request / Response)：互联网对话的基本单位，一问一答。
- HTTP / HTTPS：客户端和服务端约定的说话格式；S 表示加密。
- URL：一个资源在互联网上的完整地址。
- IP 地址 (IP Address)：每台联网设备的数字门牌号。
- 域名 (Domain)：给 IP 起的人类能记住的名字，按年付费租用。
- DNS：把域名翻译成 IP 的全球电话簿；A/CNAME/TXT/MX 是常见记录类型。
- CDN：把文件复制到全球各地节点、就近分发的网络。
- 缓存 (Cache)：把结果暂存起来下次直接用，快，但可能"旧"。

前端与后端

- 前端 (Frontend)：跑在用户设备上的那一半，用户看得见也改得了。
- 后端 (Backend)：跑在服务器上的那一半，掌管规则与数据。
- HTML / CSS / JavaScript：页面的骨架 / 皮肤 / 动作。
- 框架 (Framework)：一套现成的搭建方式和规矩，如 React。
- React：把界面拆成可复用"积木"（组件）来拼的前端框架。
- 组件 (Component)：界面积木，一块可复用的界面单元。
- 全栈 (Full-stack)：前后端都做，AI 时代你我默认就是全栈。

数据与存储

- 数据库 (Database)：专门负责"记住"的软件。
- 表 / 行 / 列 (Table / Row / Column)：数据的抽屉 / 一条记录 / 一个属性。
- SQL：和数据库说话的语言，核心是查增改删四个动词。
- Postgres：最流行的开源关系型数据库，Supabase 的地基。
- Supabase：托管好的 Postgres，附送登录、存储和现成 API。

- 迁移 (Migration) : 把每次表结构变更写成脚本, 让数据库历史可追溯、可重放。
- Schema: 数据库的整体结构设计, 有哪些表, 各有哪些列。
- RLS (Row Level Security) : 行级权限, 规定谁能看、能改哪些行。不开约等于公开。
- Auth: 登录与身份系统。永远用现成的, 别自己造。

API 与集成

- API: 软件为其他软件开的服务窗口。
- REST: 最流行的 API 风格, URL 是名词, HTTP 方法是动词。
- 端点 (Endpoint) : API 的一个具体窗口, 如 `GET /certificates/123`。
- JSON: 软件之间传数据的通用格式, 名字: 值的组合。
- API Key: 调用 API 的通行证, 属于秘密。
- Token: 广义的"凭证/令牌"。注意 AI 语境里另有含义, 指文本的计量单位。
- Webhook: 反向的 API, 你留个网址, 对方那边一有事就主动来敲门通知你。

云与部署

- 云 (Cloud) : 别人机房里租给你的电脑和服务。
- VPS / 服务器: 租的一整台永远开机的机器, 全部自管。
- Serverless: 不用管服务器的运行方式, 函数被访问才唤醒, 按次计费。
- Function / Edge Function: Serverless 里那段"被叫醒才干活"的代码。
- Vercel: 前端全球分发 + 后端函数托管 + 连 GitHub 自动部署的平台。
- 部署 (Deploy) : 把代码放到全世界能访问的地方跑起来。
- 构建 (Build) : 把源代码翻译、打包成可上线成品的过程。
- localhost: 你自己这台电脑; 只有你访问得到。
- 开发 / 预览 / 生产环境 (Dev / Preview / Production) : 私人试验田 / 每次改动的临时预览网址 / 真实用户用的正式版。
- 回滚 (Rollback) : 一键退回上一个正常版本。
- CI/CD: push 后自动测试、自动部署的流水线的统称。

版本控制

- Git: 代码的时间机器。
- GitHub: Git 仓库的云端家园与协作社区。
- 仓库 (Repository / Repo) : 一个项目的全部代码和全部历史。
- commit: 一个存档点, 附一句改动说明。

- `branch / merge`：开平行宇宙试新想法 / 把宇宙合回主线。
- `push / pull / clone`：上传存档 / 拉取存档 / 整仓复制。
- `PR (Pull Request)`：“我改好了，请过目后合并”的正式流程。
- `.gitignore`：告诉 Git“这些文件别拍进存档”的清单，`.env` 必须在列。

依赖与工具

- 库 / 包 (Library / Package)：别人写好打磨好的轮子。
- `npm / pip`：JavaScript / Python 世界的包管理器。
- 依赖 (Dependency)：你的项目用到的所有外部包。
- `package.json / lock` 文件：购物清单 / 把精确版本钉死的清单。
- `node_modules`：按清单搬回家的货，巨大而正常，可随时重装。
- 终端 (Terminal / CLI)：用命令和电脑对话的界面。
- 日志 (Log)：程序运行过程的流水账，排查问题的第一现场。
- 环境变量 (Environment Variable)：放在代码外面的配置与秘密口袋。
- `.env`：本地集中存放环境变量的文件，永不进 Git。

AI 协作

- 上下文 (Context)：AI 当前“视野”里的全部信息；有限，会被挤出去。
- 提示词 (Prompt)：你对 AI 说的话；质量决定产出。
- Agent：能自己计划、执行、观察、再计划的 AI 工作方式，Claude Code 即是。
- 幻觉 (Hallucination)：AI 一本正经的胡说。防它的办法在第 14 章。

我们的世界

- 区块链 (Blockchain)：一本全球共同记账、写上去就改不了的公共账本。
 - 智能合约 (Smart Contract)：部署到链上就无法修改的程序，相当于刻在石头上的后端，部署前的谨慎要乘一百。
 - 铸造 (Mint)：在链上创造一个新的凭证/资产，不可逆操作的典型代表。
-

附录 B · 解剖我们自己的项目

全书的概念此刻正拼在一起，跑着我们自己的东西。以 EchoForge 为例，把这张地图走一遍（以实际代码库为准，欢迎修订）。

EchoForge 的技术栈是四大件，Cloudflare + Vercel + Supabase + DO。走一遍用户的旅程，你会看到它们各守一关。

用户打开浏览器，输入我们的域名。第一关是 Cloudflare。它查自己的电话簿（DNS，第 6 章）告诉浏览器该去哪，同时兼任就近分发的调度员和挡掉恶意流量的保安（CDN 与防护）。第二关是 Vercel（第 5 章）。它送出前端，也就是 React 页面（第 2 章），并用 Serverless 函数处理偶尔醒一次的后端请求。第三关是 Supabase。那里住着 Postgres 数据库（第 3 章），存着用户和凭证的每一行数据，由 Auth 管身份，RLS 管权限（第 15 章）。第四关是 DO（DigitalOcean）上那台云服务器，第 5 章"第二层"的现实版，一台我们自己租的、永远醒着的机器，负责那些必须持续运行的活儿。穿插其间，发邮件时后端去调 Resend 的 API，钥匙装在环境变量口袋里（第 4、9 章）。最特殊的一环在链上，真正的凭证被铸造在 Cardano 区块链上，由智能合约守规矩。那是一段部署后永不能改的后端，也是我们最重要的红线（第 12 章）。所有代码住在 GitHub（第 8 章），每次 push 触发构建与部署流水线（第 11 章）。每一层的具体分工，以实际代码库为准，这正是下面这个作业要你亲手核对的。

还有另一个世界，ForgeCard。那台插着 NFC 读卡器的树莓派，就是我们自己客厅里一台永远醒着的小服务器。同样的原理，程序、依赖、日志、调试，只是它的"用户请求"来自一张靠近的卡片，而不是浏览器。

留一个好作业。和 AI 一起，把上面这段话画成一张真正的架构图，标出每个箭头对应本书哪一章。画完那天，这本书就读通了。

附录 C · 红线卡（危险动作清单）

可以打印出来贴在桌前。看到 AI 要做这些事时，停下，人来确认。

- `rm -rf` 或任何批量删除。终端删除没有回收站。
- `git push --force`。会覆盖云端历史，多人协作时是灾难。
- 一切触碰"已部署合约"的改动。链上的东西改不了，只能配合它。
- 直接操作生产数据库（删表、清数据、跑大范围 UPDATE）。先备份，先在预览环境试。
- 给真实用户群发（邮件、通知）。先发给自己，检查无误再放行。
- 花真钱的操作（铸造、转账、购买服务）。金额、地址、次数，逐项人眼核对。
- 泄露的 Key。只作废换新，光删代码没用，Git 历史里它还在。
- 改域名 DNS 记录。写错一条，全站失联。改前截图留底。

规则只有一句，可逆的事，放手让 AI 跑；不可逆的事，人是最后一道闸。

第一册《看懂地图》v1.2 · 完 下一册《上手工具》，把手放到方向盘上。